

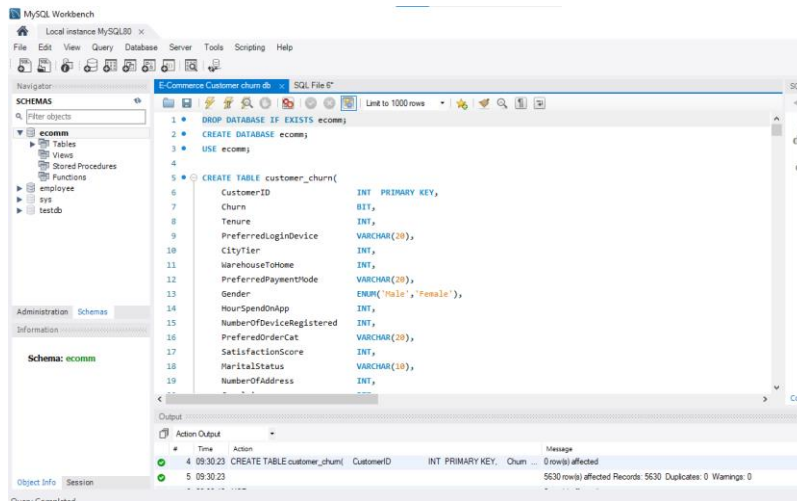
MODULE END ASSIGNMENT - MYSQL
E-Commerce Customer Churn Analysis

BY
ATHIRA SASEENDRAN

INTRODUCTION

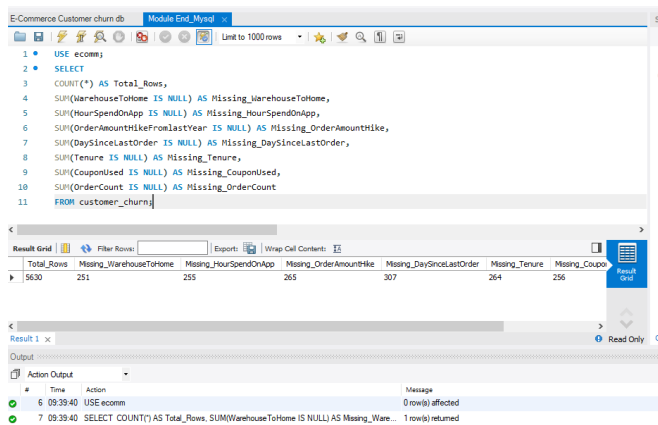
This project focuses on analysing an e-commerce customer churn dataset using MySQL. The project involves data cleaning, data transformation, and data exploration to understand customer behaviour and churn patterns. SQL queries and functions were used to handle missing values, correct inconsistencies, transform the data, and answer the required analytical questions.

1. DATA CLEANING



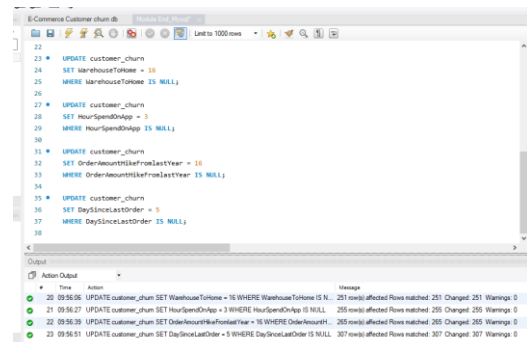
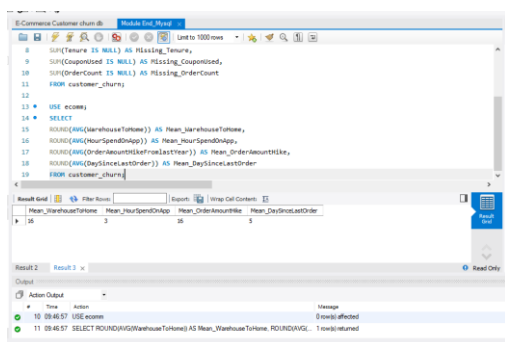
a) Handling Missing Values

The dataset was checked for missing values before performing the cleaning operations.



l) Mean Imputation

Missing values in the required numerical columns (WarehouseToHome, HourSpendOnApp, OrderAmountHikeFromlastYear, DaySinceLastOrder) were replaced using their mean values.



II) Mode Imputation

Missing values in the required categorical/count columns (Tenure, CouponUsed, OrderCount) were replaced using the most frequent value (mode).

Tensure

```
32 SET OrderCountNullToLastYear = 10
33 WHERE OrderCountNullToLastYear IS NULL;
34
35 * UPDATE customer_churn
36 SET DaysSinceLastOrder = 1
37 WHERE DaysSinceLastOrder IS NULL;
38
39 SELECT Tenure
40 FROM customer_churn
41 WHERE Tenure IS NOT NULL
42 GROUP BY Tenure
43 ORDER BY COUNT(*) DESC
44 LIMIT 1;
45
46 * UPDATE customer_churn
47 SET Tenure = 1
48 WHERE Tenure IS NULL;
```

Coupon used

```
43 WHERE Tenure IS NOT NULL;
44 GROUP BY Tenure
45 ORDER BY COUNT(*) DESC
46 LIMIT 1;
47
48 * UPDATE customer_churn
49 SET Tenure = 1
50 WHERE Tenure IS NULL;
51
52 SELECT CouponUsed
53 FROM customer_churn
54 WHERE CouponUsed IS NOT NULL
55 GROUP BY CouponUsed
56 ORDER BY COUNT(*) DESC
57 LIMIT 1;
58
59 * UPDATE customer_churn
60 SET CouponUsed = 1
61 WHERE CouponUsed IS NULL;
```

Order count

```
51 WHERE CouponUsed IS NOT NULL;
52 GROUP BY CouponUsed
53 ORDER BY COUNT(*) DESC
54 LIMIT 1;
55
56 * UPDATE customer_churn
57 SET CouponUsed = 1
58 WHERE CouponUsed IS NULL;
59
60 SELECT OrderCount
61 FROM customer_churn
62 WHERE OrderCount IS NOT NULL
63 GROUP BY OrderCount
64 ORDER BY COUNT(*) DESC
65 LIMIT 1;
66
67 * UPDATE customer_churn
68 SET OrderCount = 2
69 WHERE OrderCount IS NULL;
```

III) Remove Outliers

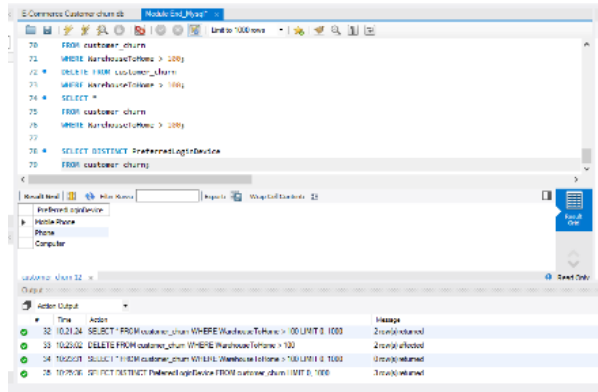
Customers with WarehouseToHome values greater than 100 were identified and removed from the dataset.

```
62 GROUP BY OrderCount
63 ORDER BY COUNT(*) DESC
64 LIMIT 1;
65
66 * UPDATE customer_churn
67 SET OrderCount = 2
68 WHERE OrderCount IS NULL;
69
70 SELECT *
71 FROM customer_churn
72 WHERE WarehouseToHome > 100;
```

```
67 WHERE OrderCount IS NULL;
68
69 * SELECT *
70 FROM customer_churn
71 WHERE WarehouseToHome > 100;
72
73 DELETE FROM customer_churn
74 WHERE WarehouseToHome > 100;
75
76 SELECT *
77 FROM customer_churn
78 WHERE WarehouseToHome > 100;
```

b) Dealing with Inconsistencies:

Inconsistent values in login device, order category and payment mode were replaced with consistent names.

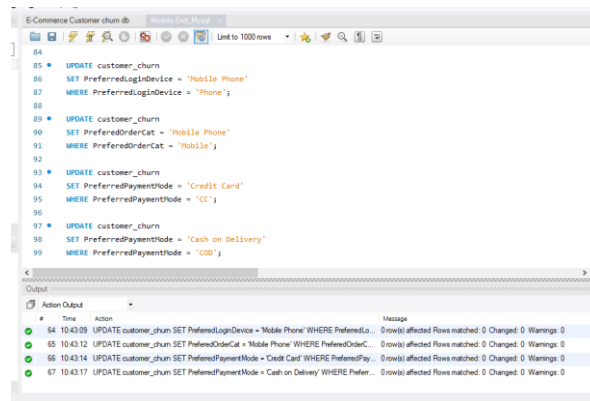


The screenshot shows a SQL IDE window titled "E-Commerce Customer churn db" with a "Module End_Mysql" tab. The SQL editor contains a query to delete records from the 'customer_churn' table where 'WarehouseToHome' is greater than 100. The query is as follows:

```
70 FROM customer_churn;
71 WHERE WarehouseToHome > 100;
72 DELETE FROM customer_churn;
73 WHERE WarehouseToHome > 100;
74 SELECT *
75 FROM customer_churn;
76 WHERE WarehouseToHome > 100;
77
78 SELECT DISTINCT PreferredLoginDevice
79 FROM customer_churn;
```

The Results panel shows the execution of the query, with a message indicating that 2 rows were deleted.

Time	Action	Message
10:21:24	SELECT * FROM customer_churn WHERE WarehouseToHome > 100 LIMIT 0, 1000	2 row(s) returned
10:21:25	DELETE FROM customer_churn WHERE WarehouseToHome > 100	2 row(s) affected
10:21:26	SELECT * FROM customer_churn WHERE WarehouseToHome > 100 LIMIT 0, 1000	0 row(s) returned
10:21:26	SELECT DISTINCT PreferredLoginDevice FROM customer_churn LIMIT 0, 1000	3 row(s) returned

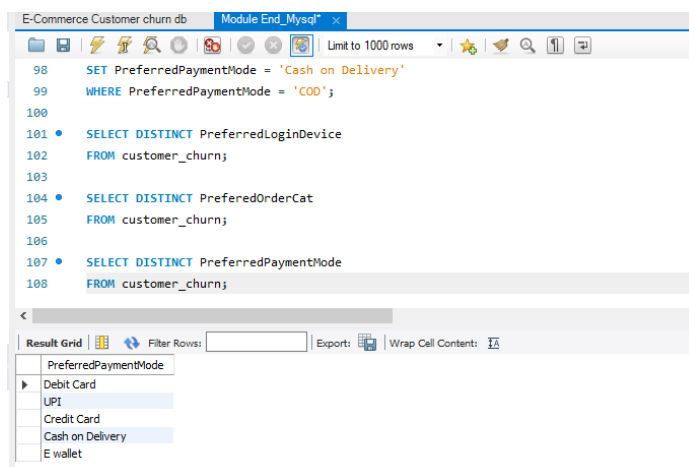


The screenshot shows a SQL IDE window titled "E-Commerce Customer churn db" with a "Module End_Mysql" tab. The SQL editor contains a series of UPDATE queries to replace inconsistent values in the 'customer_churn' table. The queries are as follows:

```
84
85 UPDATE customer_churn
86 SET PreferredLoginDevice = 'Mobile Phone'
87 WHERE PreferredLoginDevice = 'Phone';
88
89 UPDATE customer_churn
90 SET PreferredOrderCat = 'Mobile Phone'
91 WHERE PreferredOrderCat = 'Mobile';
92
93 UPDATE customer_churn
94 SET PreferredPaymentMode = 'Credit Card'
95 WHERE PreferredPaymentMode = 'CC';
96
97 UPDATE customer_churn
98 SET PreferredPaymentMode = 'Cash on Delivery'
99 WHERE PreferredPaymentMode = 'COD';
```

The Results panel shows the execution of the queries, with messages indicating the number of rows affected and warnings.

Time	Action	Message
10:43:08	UPDATE customer_churn SET PreferredLoginDevice = 'Mobile Phone' WHERE PreferredLoginDevice = 'Phone'	0 row(s) affected Rows matched: 0 Changed: 0 Warnings: 0
10:43:12	UPDATE customer_churn SET PreferredOrderCat = 'Mobile Phone' WHERE PreferredOrderCat = 'Mobile'	0 row(s) affected Rows matched: 0 Changed: 0 Warnings: 0
10:43:14	UPDATE customer_churn SET PreferredPaymentMode = 'Credit Card' WHERE PreferredPaymentMode = 'CC'	0 row(s) affected Rows matched: 0 Changed: 0 Warnings: 0
10:43:17	UPDATE customer_churn SET PreferredPaymentMode = 'Cash on Delivery' WHERE PreferredPaymentMode = 'COD'	0 row(s) affected Rows matched: 0 Changed: 0 Warnings: 0



The screenshot shows a SQL IDE window titled "E-Commerce Customer churn db" with a "Module End_Mysql" tab. The SQL editor contains a series of SELECT queries to retrieve distinct values for 'PreferredLoginDevice', 'PreferredOrderCat', and 'PreferredPaymentMode' from the 'customer_churn' table. The queries are as follows:

```
98 SET PreferredPaymentMode = 'Cash on Delivery'
99 WHERE PreferredPaymentMode = 'COD';
100
101 SELECT DISTINCT PreferredLoginDevice
102 FROM customer_churn;
103
104 SELECT DISTINCT PreferredOrderCat
105 FROM customer_churn;
106
107 SELECT DISTINCT PreferredPaymentMode
108 FROM customer_churn;
```

The Results panel shows the execution of the queries, with a message indicating the number of rows affected and warnings.

Time	Action	Message
10:43:08	UPDATE customer_churn SET PreferredLoginDevice = 'Mobile Phone' WHERE PreferredLoginDevice = 'Phone'	0 row(s) affected Rows matched: 0 Changed: 0 Warnings: 0
10:43:12	UPDATE customer_churn SET PreferredOrderCat = 'Mobile Phone' WHERE PreferredOrderCat = 'Mobile'	0 row(s) affected Rows matched: 0 Changed: 0 Warnings: 0
10:43:14	UPDATE customer_churn SET PreferredPaymentMode = 'Credit Card' WHERE PreferredPaymentMode = 'CC'	0 row(s) affected Rows matched: 0 Changed: 0 Warnings: 0
10:43:17	UPDATE customer_churn SET PreferredPaymentMode = 'Cash on Delivery' WHERE PreferredPaymentMode = 'COD'	0 row(s) affected Rows matched: 0 Changed: 0 Warnings: 0

The Results Grid shows the following data:

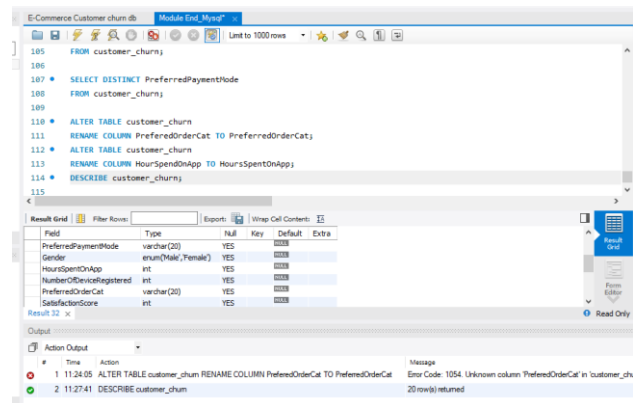
PreferredPaymentMode
Debit Card
UPI
Credit Card
Cash on Delivery
E wallet

2. DATA TRANSFORMATIONS

a) Rename Columns

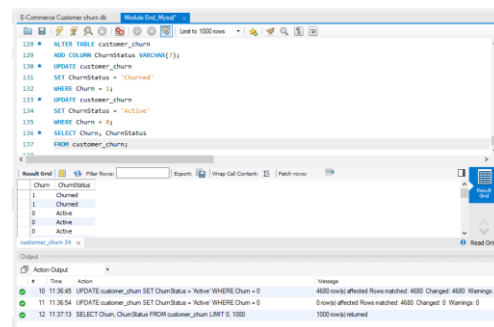
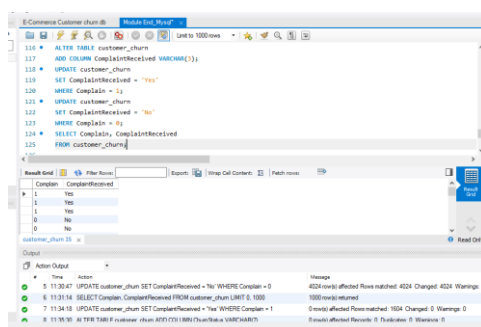
Renamed the column "PreferedOrderCat" to "PreferredOrderCat".

Renamed the column "HourSpendOnApp" to "HoursSpentOnApp".



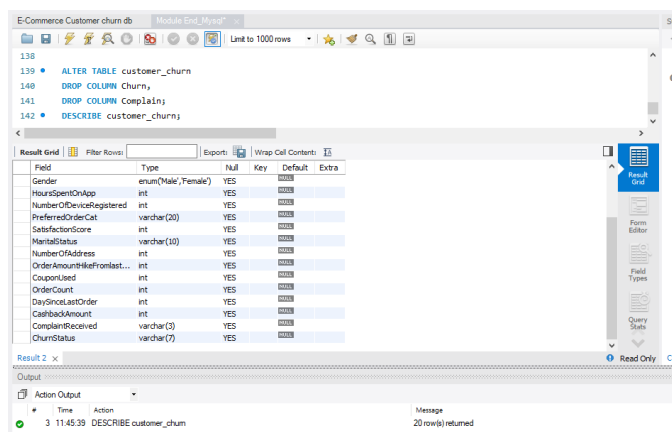
b) Create New Columns

ComplaintReceived and ChurnStatus columns were created to represent complaint and churn information clearly.



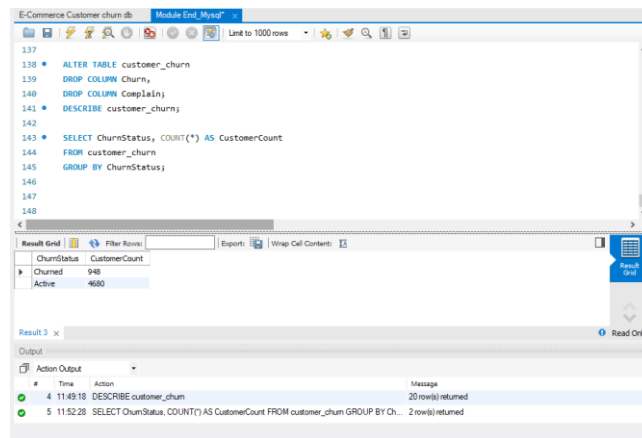
c) Remove Original Columns

The original Churn and Complain columns were removed after the new status columns were created.



3. DATA EXPLORATION AND ANALYSIS

- I. The number of active and churned customers was calculated.



```
137
138 • ALTER TABLE customer_churn
139 DROP COLUMN Churn;
140 DROP COLUMN Complains;
141 • DESCRIBE customer_churn;
142
143 • SELECT ChurnStatus, COUNT(*) AS CustomerCount
144 FROM customer_churn
145 GROUP BY ChurnStatus;
146
147
148
```

ChurnStatus	CustomerCount
Churned	948
Active	4680

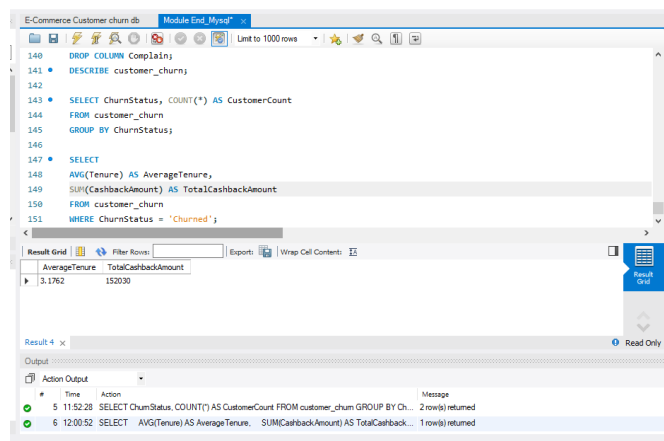
Result 3 x

Output

Action Output

#	Time	Action	Message
4	11:45:18	DESCRIBE customer_churn	20 row(s) returned
5	11:52:28	SELECT ChurnStatus, COUNT(*) AS CustomerCount FROM customer_churn GROUP BY Ch...	2 row(s) returned

- II. The average tenure and total cashback amount of churned customers were calculated.



```
140 DROP COLUMN Complains;
141 • DESCRIBE customer_churn;
142
143 • SELECT ChurnStatus, COUNT(*) AS CustomerCount
144 FROM customer_churn
145 GROUP BY ChurnStatus;
146
147 • SELECT
148 AVG(Tenure) AS AverageTenure,
149 SUM(CashbackAmount) AS TotalCashbackAmount
150 FROM customer_churn
151 WHERE ChurnStatus = 'Churned';
152
```

AverageTenure	TotalCashbackAmount
3.1762	152030

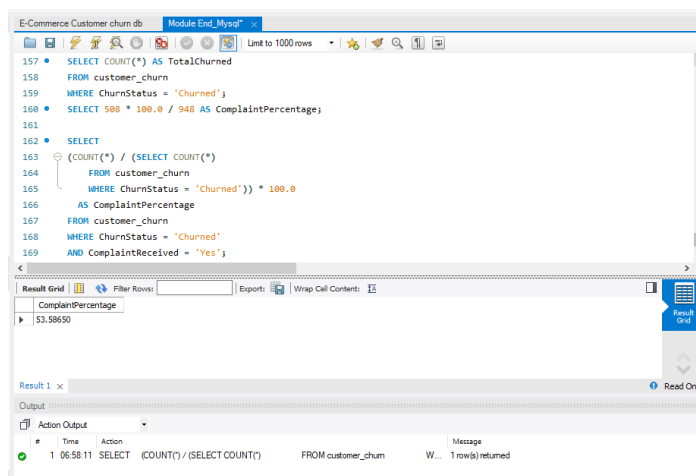
Result 4 x

Output

Action Output

#	Time	Action	Message
5	11:52:28	SELECT ChurnStatus, COUNT(*) AS CustomerCount FROM customer_churn GROUP BY Ch...	2 row(s) returned
6	12:00:52	SELECT AVG(Tenure) AS AverageTenure, SUM(CashbackAmount) AS TotalCashback...	1 row(s) returned

- III. The percentage of churned customers who made complaints was calculated.



```
157 • SELECT COUNT(*) AS TotalChurned
158 FROM customer_churn
159 WHERE ChurnStatus = 'Churned';
160 • SELECT 508 * 100.0 / 948 AS ComplaintPercentage;
161
162 • SELECT
163 (COUNT(*) / (SELECT COUNT(*)
164 FROM customer_churn
165 WHERE ChurnStatus = 'Churned'))) * 100.0
166 AS ComplaintPercentage
167 FROM customer_churn
168 WHERE ChurnStatus = 'Churned'
169 AND ComplaintReceived = 'Yes';
170
```

ComplaintPercentage
53.58650

Result 1 x

Output

Action Output

#	Time	Action	Message
1	06:58:11	SELECT (COUNT(*) / (SELECT COUNT(*) FROM customer_churn WHERE ChurnStatus = 'Churned')) * 100.0 AS ComplaintPercentage FROM customer_churn WHERE ChurnStatus = 'Churned' AND ComplaintReceived = 'Yes';	1 row(s) returned

- IV. The city tier with the highest number of churned Laptop & Accessory customers was identified.

```
167 FROM customer_churn
168 WHERE ChurnStatus = 'Churned'
169 AND ComplaintReceived = 'Yes';
170
171 • SELECT CityTier, COUNT(*) AS ChurnedCustomers
172 FROM customer_churn
173 WHERE ChurnStatus = 'Churned'
174 AND PreferredOrderCat = 'Laptop & Accessory'
175 GROUP BY CityTier
176 ORDER BY ChurnedCustomers DESC
177 LIMIT 1;
178
179
```

Result Grid

CityTier	ChurnedCustomers
3	150

Result 2 x

Output

Action Output

#	Time	Action	Message
1	06:58:11	SELECT (COUNT(*) / (SELECT COUNT(*) FROM customer_churn WHERE ChurnStatus = 'Churned')) AS ChurnRate FROM customer_churn WHERE ChurnStatus = 'Churned' AND ComplaintReceived = 'Yes';	1 row(s) returned
2	07:02:16	SELECT CityTier, COUNT(*) AS ChurnedCustomers FROM customer_churn WHERE ChurnStatus = 'Churned' AND PreferredOrderCat = 'Laptop & Accessory' GROUP BY CityTier ORDER BY ChurnedCustomers DESC LIMIT 1;	1 row(s) returned

- V. The most preferred payment mode among active customers was identified.

```
174 AND PreferredOrderCat = 'Laptop & Accessory'
175 GROUP BY CityTier
176 ORDER BY ChurnedCustomers DESC
177 LIMIT 1;
178
179 • SELECT PreferredPaymentMode, COUNT(*) AS CustomerCount
180 FROM customer_churn
181 WHERE ChurnStatus = 'Active'
182 GROUP BY PreferredPaymentMode
183 ORDER BY CustomerCount DESC
184 LIMIT 1;
185
186
```

Result Grid

PreferredPaymentMode	CustomerCount
Debit Card	1956

Result 3 x

Output

Action Output

#	Time	Action	Message
2	07:02:16	SELECT CityTier, COUNT(*) AS ChurnedCustomers FROM customer_churn WHERE ChurnStatus = 'Churned' AND PreferredOrderCat = 'Laptop & Accessory' GROUP BY CityTier ORDER BY ChurnedCustomers DESC LIMIT 1;	1 row(s) returned
3	07:05:08	SELECT PreferredPaymentMode, COUNT(*) AS CustomerCount FROM customer_churn WHERE ChurnStatus = 'Active' GROUP BY PreferredPaymentMode ORDER BY CustomerCount DESC LIMIT 1;	1 row(s) returned

- VI. The total order amount hike from last year was calculated for single customers who prefer mobile phones.

```
179 • SELECT PreferredPaymentMode, COUNT(*) AS CustomerCount
180 FROM customer_churn
181 WHERE ChurnStatus = 'Active'
182 GROUP BY PreferredPaymentMode
183 ORDER BY CustomerCount DESC
184 LIMIT 1;
185
186 • SELECT SUM(OrderAmountHikeFromLastYear) AS TotalOrderAmountHike
187 FROM customer_churn
188 WHERE MaritalStatus = 'Single'
189 AND PreferredOrderCat = 'Mobile Phone';
190
191
```

Result Grid

TotalOrderAmountHike
12177

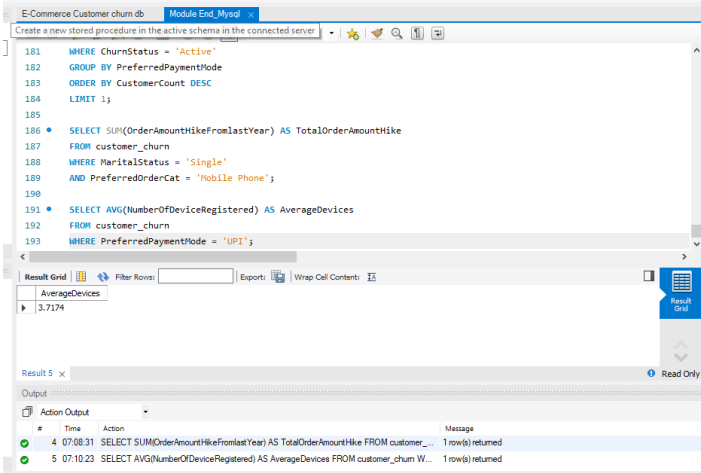
Result 4 x

Output

Action Output

#	Time	Action	Message
3	07:05:08	SELECT PreferredPaymentMode, COUNT(*) AS CustomerCount FROM customer_churn WHERE ChurnStatus = 'Active' GROUP BY PreferredPaymentMode ORDER BY CustomerCount DESC LIMIT 1;	1 row(s) returned
4	07:08:31	SELECT SUM(OrderAmountHikeFromLastYear) AS TotalOrderAmountHike FROM customer_churn WHERE MaritalStatus = 'Single' AND PreferredOrderCat = 'Mobile Phone';	1 row(s) returned

VII. The average number of registered devices was calculated for customers using UPI.



The screenshot shows a SQL query in a database client. The query is as follows:

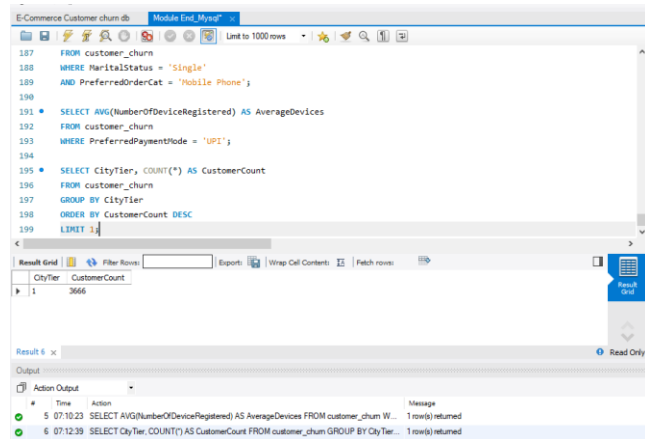
```
181 WHERE ChurnStatus = 'Active'
182 GROUP BY PreferredPaymentMode
183 ORDER BY CustomerCount DESC
184 LIMIT 1;
185
186 SELECT SUM(OrderAmountHikeFromLastYear) AS TotalOrderAmountHike
187 FROM customer_churn
188 WHERE MaritalStatus = 'Single'
189 AND PreferredOrderCat = 'Mobile Phone';
190
191 SELECT AVG(NumberOfDeviceRegistered) AS AverageDevices
192 FROM customer_churn
193 WHERE PreferredPaymentMode = 'UPI';
```

The result grid shows one row with the value 3.7174 for AverageDevices.

AverageDevices
3.7174

The output pane shows the execution of the query, with a message indicating that 1 row(s) returned.

VIII. The city tier containing the highest number of customers was identified.



The screenshot shows a SQL query in a database client. The query is as follows:

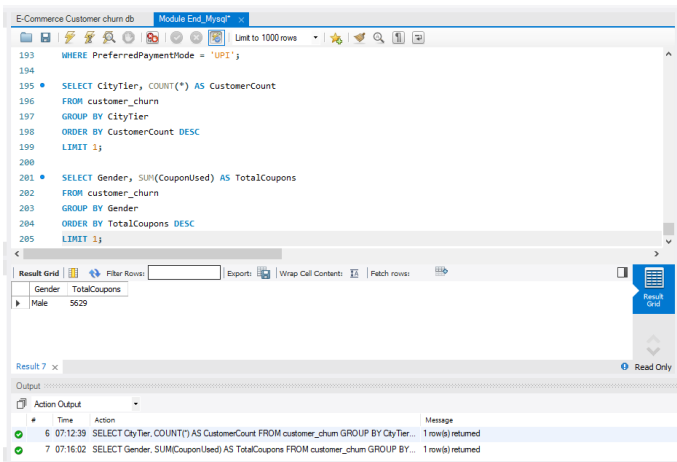
```
187 FROM customer_churn
188 WHERE MaritalStatus = 'Single'
189 AND PreferredOrderCat = 'Mobile Phone';
190
191 SELECT AVG(NumberOfDeviceRegistered) AS AverageDevices
192 FROM customer_churn
193 WHERE PreferredPaymentMode = 'UPI';
194
195 SELECT CityTier, COUNT(*) AS CustomerCount
196 FROM customer_churn
197 GROUP BY CityTier
198 ORDER BY CustomerCount DESC
199 LIMIT 1;
```

The result grid shows one row with the value 3666 for CustomerCount.

CityTier	CustomerCount
1	3666

The output pane shows the execution of the query, with a message indicating that 1 row(s) returned.

IX. The gender with the highest total number of coupons used was identified.



The screenshot shows a SQL query in a database client. The query is as follows:

```
193 WHERE PreferredPaymentMode = 'UPI';
194
195 SELECT CityTier, COUNT(*) AS CustomerCount
196 FROM customer_churn
197 GROUP BY CityTier
198 ORDER BY CustomerCount DESC
199 LIMIT 1;
200
201 SELECT Gender, SUM(CouponUsed) AS TotalCoupons
202 FROM customer_churn
203 GROUP BY Gender
204 ORDER BY TotalCoupons DESC
205 LIMIT 1;
```

The result grid shows one row with the value 5629 for TotalCoupons.

Gender	TotalCoupons
Male	5629

The output pane shows the execution of the query, with a message indicating that 1 row(s) returned.

- X. The customer count and maximum app usage hours were calculated for each preferred order category.

```
201 • SELECT Gender, SUM(CouponUsed) AS TotalCoupons
202 FROM customer_churn
203 GROUP BY Gender
204 ORDER BY TotalCoupons DESC
205 LIMIT 1;

207 • SELECT PreferredOrderCat,
208 COUNT(*) AS CustomerCount,
209 MAX(HoursSpentOnApp) AS MaximumHoursSpent
210 FROM customer_churn
211 GROUP BY PreferredOrderCat;
```

PreferredOrderCat	CustomerCount	MaximumHoursSpent
Laptop & Accessory	2050	5
Mobile Phone	2076	5
Others	264	4
Fashion	826	5
Grocery	410	4

Result 8 x

Output

#	Time	Action	Message
7	07:16:02	SELECT Gender, SUM(CouponUsed) AS TotalCoupons FROM customer_churn GROUP BY...	1 row(s) returned
8	07:17:43	SELECT PreferredOrderCat, COUNT(*) AS CustomerCount, MAX(HoursSpentOnApp)...	5 row(s) returned

- XI. The total order count was calculated for credit card users with the maximum satisfaction score.

```
213 • SELECT PreferredOrderCat,
214 COUNT(*) AS CustomerCount,
215 MAX(HoursSpentOnApp) AS MaximumHoursSpent
216 FROM customer_churn
217 GROUP BY PreferredOrderCat;

219 • SELECT SUM(OrderCount) AS TotalOrderCount
220 FROM customer_churn
221 WHERE PreferredPaymentMode = 'Credit Card'
222 AND SatisfactionScore = (
223 SELECT MAX(SatisfactionScore)
224 FROM customer_churn);
```

TotalOrderCount
1122

Result 9 x

Output

#	Time	Action	Message
8	07:17:43	SELECT PreferredOrderCat, COUNT(*) AS CustomerCount, MAX(HoursSpentOnApp)...	5 row(s) returned
9	07:20:42	SELECT SUM(OrderCount) AS TotalOrderCount FROM customer_churn WHERE PreferredP...	1 row(s) returned

- XII. The average satisfaction score was calculated for customers who made complaints.

```
219 FROM customer_churn
220 GROUP BY PreferredOrderCat;

223 • SELECT SUM(OrderCount) AS TotalOrderCount
224 FROM customer_churn
225 WHERE PreferredPaymentMode = 'Credit Card'
226 AND SatisfactionScore = (
227 SELECT MAX(SatisfactionScore)
228 FROM customer_churn);

229 • SELECT AVG(SatisfactionScore) AS AverageSatisfactionScore
230 FROM customer_churn
231 WHERE ComplaintReceived = 'Yes';
```

AverageSatisfactionScore
2.9988

Result 10 x

Output

#	Time	Action	Message
9	07:20:42	SELECT SUM(OrderCount) AS TotalOrderCount FROM customer_churn WHERE PreferredP...	1 row(s) returned
10	07:22:42	SELECT AVG(SatisfactionScore) AS AverageSatisfactionScore FROM customer_churn WH...	1 row(s) returned

- XIII. Preferred order categories were listed for customers who used more than five coupons.

```
215 WHERE PreferredPaymentMode = 'Credit Card'
216 AND SatisfactionScore = (
217     SELECT MAX(SatisfactionScore)
218     FROM customer_churn)
219
220 • SELECT AVG(SatisfactionScore) AS AverageSatisfactionScore
221 FROM customer_churn
222 WHERE ComplaintReceived = 'Yes';
223
224 • SELECT PreferredOrderCat, COUNT(*) AS CustomerCount
225 FROM customer_churn
226 WHERE CouponUsed > 5
227 GROUP BY PreferredOrderCat;
```

PreferredOrderCat	CustomerCount
Others	28
Laptop & Accessory	99
Fashion	89
Mobile Phone	45
Grocery	42

Result 11 x

Output

#	Time	Action	Message
10	07:22:42	SELECT AVG(SatisfactionScore) AS AverageSatisfactionScore FROM customer_churn WHERE...	1 row(s) returned
11	07:24:17	SELECT PreferredOrderCat, COUNT(*) AS CustomerCount FROM customer_churn WHERE...	5 row(s) returned

- XIV. The three preferred order categories with the highest average cashback were identified.

```
221 FROM customer_churn
222 WHERE ComplaintReceived = 'Yes';
223
224 • SELECT PreferredOrderCat, COUNT(*) AS CustomerCount
225 FROM customer_churn
226 WHERE CouponUsed > 5
227 GROUP BY PreferredOrderCat;
228
229 • SELECT PreferredOrderCat, AVG(CashbackAmount) AS AverageCashback
230 FROM customer_churn
231 GROUP BY PreferredOrderCat
232 ORDER BY AverageCashback DESC
233 LIMIT 3;
```

PreferredOrderCat	AverageCashback
Others	304.4545
Grocery	266.2366
Fashion	210.4021

Result 12 x

Output

#	Time	Action	Message
11	07:24:17	SELECT PreferredOrderCat, COUNT(*) AS CustomerCount FROM customer_churn WHERE...	5 row(s) returned
12	07:25:40	SELECT PreferredOrderCat, AVG(CashbackAmount) AS AverageCashback FROM customer...	3 row(s) returned

- XV. Payment modes were identified where the average tenure was at least 10 months and total orders were more than 500.

```
227 GROUP BY PreferredOrderCat;
228
229 • SELECT PreferredOrderCat, AVG(CashbackAmount) AS AverageCashback
230 FROM customer_churn
231 GROUP BY PreferredOrderCat
232 ORDER BY AverageCashback DESC
233 LIMIT 3;
234
235 • SELECT PreferredPaymentMode, AVG(Tenure) AS AverageTenure,
236 SUM(OrdersCount) AS TotalOrders
237 FROM customer_churn
238 GROUP BY PreferredPaymentMode
239 HAVING AVG(Tenure) >= 10 AND SUM(OrdersCount) > 500;
```

PreferredPaymentMode	AverageTenure	TotalOrders
E-wallet	10.1954	1849

Result 16 x

Output

#	Time	Action	Message
15	07:30:28	SELECT PreferredPaymentMode AVG(Tenure) AS AverageTenure, SUM(OrdersCount) AS To...	3 row(s) returned
16	07:34:18	SELECT PreferredPaymentMode, AVG(Tenure) AS AverageTenure, SUM(OrdersCou...	1 row(s) returned

- XVI. Customers were categorized based on their distance from the warehouse, and the churn status breakdown was displayed for each distance category.

The screenshot shows a database query editor with the following SQL query:

```
239 HAVING AVG(Tenure) >= 10 AND SUM(OrderCount) > 500;
240
241
242 SELECT
243 CASE
244 WHEN WarehouseToHome <= 5 THEN 'Very Close Distance'
245 WHEN WarehouseToHome <= 10 THEN 'Close Distance'
246 WHEN WarehouseToHome <= 15 THEN 'Moderate Distance'
247 ELSE 'Far Distance'
248 END AS DistanceCategory, ChurnStatus, COUNT(*) AS CustomerCount
249 FROM customer_churn
250 GROUP BY DistanceCategory, ChurnStatus
251 ORDER BY DistanceCategory, ChurnStatus;
```

The result grid shows the following data:

DistanceCategory	ChurnStatus	CustomerCount
Close Distance	Active	1696
Close Distance	Churned	265
Far Distance	Active	1871
Far Distance	Churned	498
Moderate Distance	Active	1106

The output section shows the following action output:

#	Time	Action	Message
16	07:34:18	SELECT PreferredPaymentMode, AVG(Tenure) AS Average Tenure, SUM(OrderCou...	1 row(s) returned
17	07:38:11	SELECT CASE WHEN WarehouseToHome <= 5 THEN 'Very Close Distance' WHEN Wa...	8 row(s) returned

- XVII. Customer details were listed for married City Tier-1 customers whose order count was above the overall average.

The screenshot shows a database query editor with the following SQL query:

```
251 ORDER BY DistanceCategory, ChurnStatus;
252
253 SELECT * FROM customer_churn
254 WHERE MaritalStatus = 'Married'
255 AND CityTier = 1
256 AND OrderCount > (
257 SELECT AVG(OrderCount)
258 FROM customer_churn
259 );
```

The result grid shows the following data:

CustomerID	Tenure	PreferredLoginDevice	CityTier	WarehouseToHome	PreferredPaymentMode	Gender	HoursSpentOnApp	NumberOfDeviceRege
50049	3	Computer	1	15	Credit Card	Male	3	3
50119	13	Mobile Phone	1	30	Cash on Delivery	Male	3	3
50132	15	Mobile Phone	1	35	Credit Card	Female	2	4
50133	13	Mobile Phone	1	8	Credit Card	Male	2	3
50152	2	Mobile Phone	1	28	Debit Card	Female	3	2
50180	12	Mobile Phone	1	6	Credit Card	Male	2	2
50187	18	Computer	1	13	Debit Card	Male	3	3
50229	10	Mobile Phone	1	12	Credit Card	Male	2	2

The output section shows the following action output:

#	Time	Action	Message
17	07:38:11	SELECT CASE WHEN WarehouseToHome <= 5 THEN 'Very Close Distance' WHEN Wa...	8 row(s) returned
18	07:40:28	SELECT * FROM customer_churn WHERE MaritalStatus = 'Married' AND CityTier = 1 AND ...	576 row(s) returned

XVIII. Create Customer Returns Table

- a) A customer_returns table was created in the ecomm database and the eight given return records were inserted.

The screenshot shows a SQL script in the 'E-Commerce Customer churn db' database. The script includes an INSERT statement to create the 'customer_returns' table and populate it with eight records. The records are as follows:

ReturnID	CustomerID	ReturnDate	RefundAmount
1001	50022	2023-01-01	2130
1002	50316	2023-01-23	2000
1003	51099	2023-02-14	2290
1004	52321	2023-03-08	2510
1005	52928	2023-03-20	3000
1006	53749	2023-04-17	1740
1007	54206	2023-04-21	3250
1008	54838	2023-04-30	1990

The 'Result Grid' shows the first five rows of the inserted data. The 'Output' pane shows the execution results: 8 rows affected for the INSERT statement and 8 rows returned for the SELECT statement.

- b) The returns table was joined with the customer churn table to display return details for churned customers who made complaint

The screenshot shows a SQL script in the 'E-Commerce Customer churn db' database. The script includes a JOIN query that combines data from the 'customer_returns' and 'customer_churn' tables. The query filters for churned customers who have received a complaint. The results are shown in the 'Result Grid' and the 'Output' pane.

ReturnID	CustomerID	ReturnDate	RefundAmount	CustomerID	Tenure	PreferredLoginDevice	CityTier	WarehouseToHome	PreferredPaymentMode
1002	50316	2023-01-23	2000	50316	0	Computer	2	29	UPI
1004	52321	2023-03-08	2510	52321	18	Mobile Phone	3	19	E wallet
1006	53749	2023-04-17	1740	53749	1	Mobile Phone	3	31	Credit Card

The 'Output' pane shows the execution results: 3 rows returned for the JOIN query.

CONCLUSION

The project successfully completed the required MySQL data cleaning, transformation and analysis tasks. Missing values were handled, inconsistent records were corrected, unnecessary outliers were removed, and the dataset was transformed into a more useful format. The analysis questions were answered using aggregate functions, grouping, filtering, subqueries and joins. A separate customer_returns table was also created and connected with the customer data to obtain the required return information.